Interfețe și Moșteniri

Interfețe marker (Marker Interfaces)

Interfețele marker sunt interfețe **fără metode**, folosite doar pentru a „marca” o clasă. Ele transmit informații către JVM sau către alte clase că obiectul are o anumită proprietate.

```
import java.io.Serializable;

public class User implements Serializable {
    String name;
}
```

Aici, Serializable este o interfață marker.

Codul verifică dacă un obiect implementează interfața marker:

```
if (obj instanceof Serializable) {
    // tratament special
}
```

Exemple din Java

- Serializable → permite serializarea obiectelor
- Cloneable → permite clonarea
- RandomAccess → indică acces rapid (liste)

Caracteristici

- Nu conțin metode
- Folosesc **polimorfismul prin tip**
- Sunt înlocuite uneori de **anotații** (@Override, etc.)

Clase adaptor (Adapter Classes)

Clasele adaptor sunt clase abstracte care implementează o interfață cu metode multiple și oferă implementări **goale** (default). Ele evită obligația de a implementa toate metodele unei interfețe.

```
import java.awt.event.*;

public abstract class MouseAdapter implements MouseListener {
    public void mouseClicked(MouseEvent e) {}
    public void mousePressed(MouseEvent e) {}
    public void mouseReleased(MouseEvent e) {}
    public void mouseEntered(MouseEvent e) {}
    public void mouseExited(MouseEvent e) {}
}
```

```
public class MyMouse extends MouseAdapter {
    @Override
    public void mouseClicked(MouseEvent e) {
        System.out.println("Click!");
    }
}
```

- Sunt **abstracte**
- Reduc codul boilerplate
- Utilizate mai ales în GUI și evenimente

Extinderea interfețelor (default, private methods)

Inițial, interfețele aveau doar metode abstracte. Din Java 8+:

- **Metode default**

Au implementare implicită.

```
interface Animal {
    default void eat() {
```

```
        System.out.println("Animal eats");
    }
}
```

- **Metode statice**

```
interface MathUtil {
    static int add(int a, int b) {
        return a + b;
    }
}
```

- **Metode private (Java 9+)**

Folosite pentru reutilizare internă:

```
interface Example {
    default void methodA() {
        helper();
    }

    private void helper() {
        System.out.println("Helper");
    }
}
```

Extinderea interfețelor

```
interface A {
    void methodA();
}

interface B extends A {
    void methodB();
}
```

Caracteristici

- Permite evoluția API-urilor fără breaking changes
- Suportă reutilizare logică
- Nu pot avea stare (fără variabile de instanță)

Prioritatea la extindere (clasă vs interfață)

Metodele din clase au prioritate față de cele din interfețe

Exemplu

```
class A {
    public void show() {
        System.out.println("Class A");
    }
}

interface B {
    default void show() {
        System.out.println("Interface B");
    }
}

class C extends A implements B {
    public static void main(String[] args) {
        new C().show();
    }
}
```

Output:

Class A

Conflict între interfețe

```
interface A {
    default void show() {}
}

interface B {
```

```
    default void show() {}
}

class C implements A, B {
    public void show() {
        A.super.show(); // trebuie specificat
    }
}
```

Reguli

1. Clasa > Interfață
2. Conflict între interfețe → trebuie rezolvat explicit

Controlul moștenirii cu sealed

Keyword introdus în Java 15 (preview), stabil din Java 17. Permite restricționarea claselor care pot extinde o clasă.

Exemplu

```
public sealed class Shape permits Circle, Rectangle {
}
final class Circle extends Shape {}
non-sealed class Rectangle extends Shape {}
```

Tipuri de clase derivate

- final → nu mai poate fi extinsă
- sealed → continuă restricția
- non-sealed → elimină restricția

Beneficii

- Control strict asupra ierarhiei
- Util pentru pattern matching
- Cod mai sigur și predictibil

Exemplu complet

```
public sealed interface Vehicle permits Car, Truck {}

final class Car implements Vehicle {}

non-sealed class Truck implements Vehicle {}
```

Enumerări (Enums)

Tipuri speciale care definesc un set fix de constante.

Exemplu simplu

```
enum Day {
    MONDAY, TUESDAY, WEDNESDAY
}
```

Cu atribute și constructor

```
enum Planet {
    EARTH(5.97), MARS(0.642);

    private double mass;

    Planet(double mass) {
        this.mass = mass;
    }

    public double getMass() {
        return mass;
    }
}
```

Cu metode

```
enum Operation {
    ADD {
        public int apply(int a, int b) { return a + b; }
    },
    SUB {
        public int apply(int a, int b) { return a - b; }
    };

    public abstract int apply(int a, int b);
}
```

Caracteristici

- Sunt implicit static și final
- Extind java.lang.Enum
- Pot avea:
  - constructori
  - metode
  - câmpuri
- Nu pot extinde alte clase

Metode utile

```
Day.valueOf("MONDAY");
Day.values();
```

Exercitii

1 — Interfață Marker (Serializable custom)

Creează o interfață marker numită:

interface Auditable {}

Creează două clase:

- User (implementează Auditable)
- Product (NU implementează)

Creează o clasă AuditService care:

- primește un obiect
- verifică dacă este instanceof Auditable
- dacă da → afișează "Audit log created"
- altfel → "Not auditable"

Bonus

Simulează un sistem de logging diferit pentru obiectele marcate.

2 — Clasă adaptor (Event System)

Creează o interfață WindowListener cu 3 functii: onOpen, onClose si OnResize.

Creează o clasă MyWindow care suprascrie doar onOpen() și afișează mesaj.

3 — Interfețe moderne (default + private)

Creează interfața Payment Processor cu functiile

- process – abtracta
- log – default – valideaza si printeaza in consola un log
- validate – private – verifica daca amount este mai mare ca 0

Toate functiile au ca parametru un int amount.

Creează:

class CreditCardPayment implements PaymentProcessor

- implementează process
- apelează metoda log

4 — Prioritate clasă vs interfață

Creează:

- clasa BasePrinter cu functia publica print
- interfata AdvancedPrinter cu functia default print
- clasa Printer care extinde BasePrinter si implementeaza AdvancedPrinter

Apelează print() și observă rezultatul

Adaugă:

- alta interfata AnotherPrinter tot cu o metoda default print
- Printer sa implementeze si pe AnotherPrinter

Rezolvă conflictul explicit.

5 — Sealed + Enum (mini sistem de transport)

Creează enum FuelType care sa fie PETROL, DIESEL sau ELECTRIC

Creează interfață sealed Vehicle care sa permita doar mostenirea de catre Car si Truck

Creează clase Car (final) si Truck (non-sealed) care sa implementeze Vehicle. Car are un parametru FuelType si Truck are un parametru capacity

4. Creează o clasă:

class VehicleService

metodă:

void describe(Vehicle v)

- dacă e Car → afișează tip combustibil
- dacă e Truck → afișează capacitate